Manual Técnico

Creador Formularios

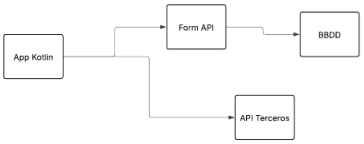
Introducción

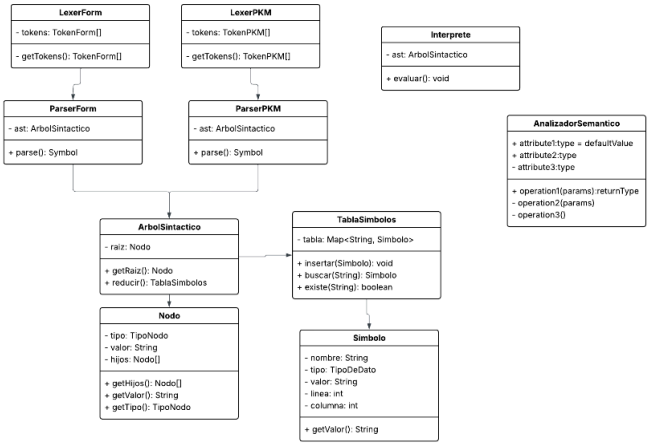
Programa escrito en kotlin que crea formularios a partir de comandos

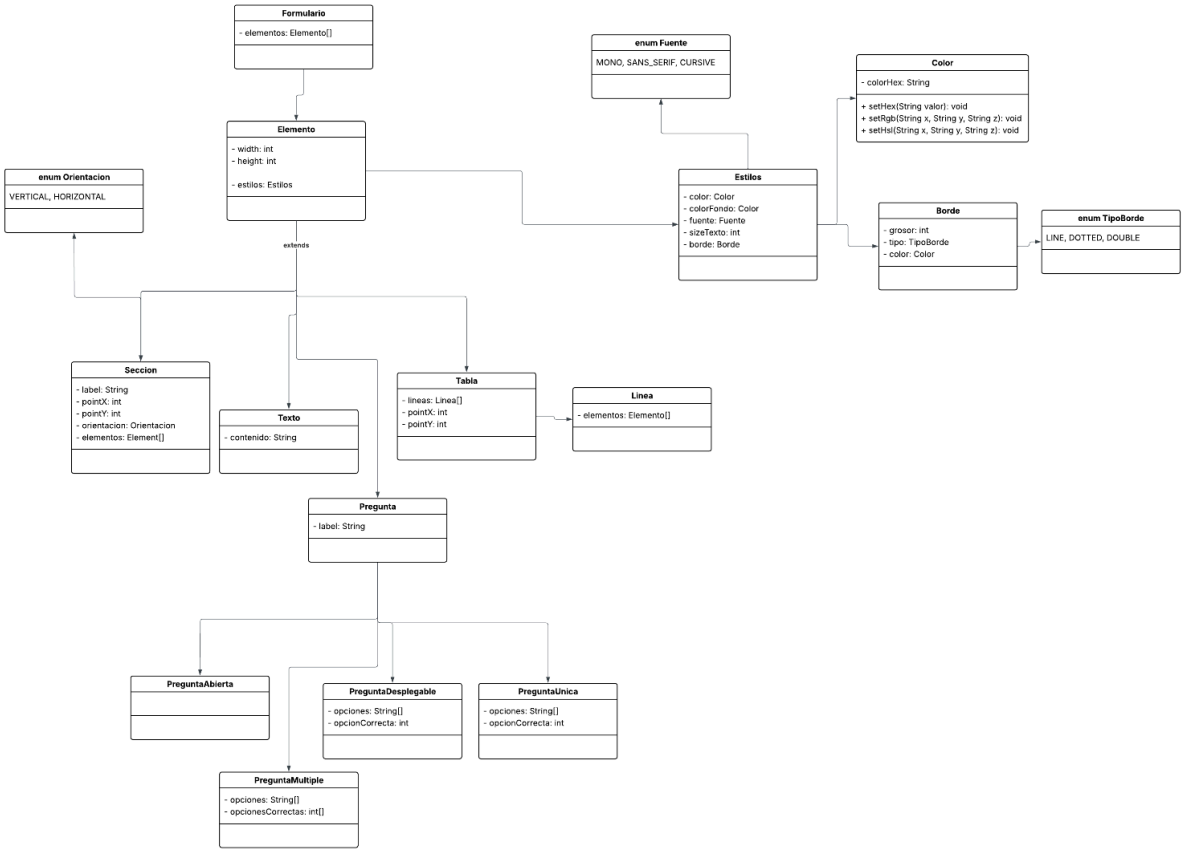
Tecnologías utilizadas

open-jdk 21
jflex 1.9.1
java cup 11b
gradle 9.2.1
kotlin 2.2.20

Diagrama de clases


App Kotlin
Form API
BBDD
API Terceros


LexerForm
- tokens: TokenForm[]
- getTokens(): TokenForm[]
LexerPKM
- tokens: TokenPKM[]
- getTokens(): TokenPKM[]
Interprete
- ast: ArbolSintactico
+ evaluar(): void
ParserForm
- ast: ArbolSintactico
+ parse(): Symbol
ParserPKM
- ast: ArbolSintactico
+ parse(): Symbol
AnalizadorSemantico
+ attribute1:type = defaultValue
+ attribute2:type
- attribute3:type
+ operation1(params):returnType
- operation2(params)
- operation3()
ArbolSintactico
- raiz: Nodo
+ getRaiz(): Nodo
+ reducir(): TablaSimbolos
TablaSimbolos
- tabla: Map<String, Simbolo>
+ insertar(Simbolo): void
+ buscar(String): Simbolo
+ existe(String): boolean
Nodo
- tipo: TipoNodo
- valor: String
- hijos: Nodo[]
+ getHijos(): Nodo[]
+ getValor(): String
+ getTipo(): TipoNodo
Simbolo
- nombre: String
- tipo: TipoDeDato
- valor: String
- linea: int
- columna: int
+ getValor(): String


Formulario
- elementos: Elemento[]
enum Fuente
MONO, SANS_SERIF, CURSIVE
Color
- colorHex: String
+ setHex(String valor): void
+ setRgb(String x, String y, String z): void
+ setHsl(String x, String y, String z): void
Elemento
- width: int
- height: int
- estilos: Estilos
enum Orientacion
VERTICAL, HORIZONTAL
Estilos
- color: Color
- colorFondo: Color
- fuente: Fuente
- sizeTexto: int
- borde: Borde
Borde
- grosor: int
- tipo: TipoBorde
- color: Color
enum TipoBorde
LINE, DOTTED, DOUBLE
extends
Seccion
- label: String
- pointX: int
- pointY: int
- orientacion: Orientacion
- elementos: Element[]
Texto
- contenido: String
Tabla
- lineas: Linea[]
- pointX: int
- pointY: int
Linea
- elementos: Elemento[]
Pregunta
- label: String
PreguntaAbierta
PreguntaDesplegable
- opciones: String[]
- opcionCorrecta: int
PreguntaUnica
- opciones: String[]
- opcionCorrecta: int
PreguntaMultiple
- opciones: String[]
- opcionesCorrectas: int[]

Archivos configuración flex

```
package com.mycompany.testlexer2;

import java_cup.runtime.Symbol;
import java.util.ArrayList;
import java.util.List;
import java.io.IOException;

%%

%xstate STRING

%class LexerForm
%unicode
%line
%column
%public
%cupsym symform
%cup
%extends Lexer

%{



private List<Token> tokens = new ArrayList<>();
private List<Token> errores = new ArrayList<>();

private boolean hayError = false;
private StringBuilder errorActual = new StringBuilder();
private int lineaErrorActual = 0;
private int columnaErrorActual = 0;

private StringBuilder bufferCadena = new StringBuilder();
private int lineaCadena = 0;
private int columnaCadena = 0;

public List<Token> getTokens(){
	return tokens;
}

public List<Token> getErrores(){
	return errores;
}

private Symbol simbolo(int type){
```

```
        return new Symbol(type, yyline, yycolumn);
}

private Symbol simbolo(int type, String value){
        return new Symbol(type, yyline, yycolumn, value);
}

private void agregarToken(TipoToken tipo){
        if(hayError){
        errorActual.append(yytext());
        lineaErrorActual = yyline + 1;
        columnaErrorActual = yycolumn + 1;
        }else{
        tokens.add(new Token(yytext(), yyline + 1, yycolumn + 1, tipo));
        }
}

private void agregarToken(String lexema, int linea, int columna, TipoToken tipo){
        tokens.add(new Token(lexema, linea, columna, tipo));
}

private void agregarError(){
        errores.add(new Token(errorActual.toString(), lineaErrorActual, columnaErrorActual,
TipoToken.ERROR));
}

private void manejarEspacio(){
        if(hayError && errorActual.length() > 0){
        hayError = false;
        agregarError();
        errorActual.setLength(0);
        }
}

private Symbol agregarTextoCadena(){
        if(bufferCadena.length() > 0){
        String texto = bufferCadena.toString();
        agregarToken(texto, lineaCadena, columnaCadena, TipoToken.CADENA);
        bufferCadena.setLength(0);
        return new Symbol(sym.CADENA, lineaCadena - 1, columnaCadena - 1, texto);
        }
        return null;
}

private Symbol registrarYRetornar(int cupSym, TipoToken tipo){
        agregarToken(tipo);
        return simbolo(cupSym);
}
```

```
private Symbol registrarYRetornar(int cupSym, TipoToken tipo, String value){
	agregarToken(tipo);
	return simbolo(cupSym, value);
}

%}

DIGITO = [0-9]
LETRA = [a-zA-Z_]
IDENT = {LETRA}({LETRA}|{DIGITO})*

ENTERO = {DIGITO}+
DECIMAL = {DIGITO}+\.{DIGITO}+

COLORHEX = #[0-9A-Fa-f]{6}
COLORRGB = rgb\(\s*{ENTERO}\s*,\s*{ENTERO}\s*,\s*{ENTERO}\s*\)

COLORHSL = \<\s*{ENTERO}\s*,\s*{ENTERO}\s*,\s*{ENTERO}\s*\>



COMENTARIO_LINEA = \$[^\n]*
COMENTARIO_BLOQUE = "/*"([^*]|\*+[^*/])*\*+"/"

%%

<STRING>{

\" {
	Symbol s = agregarTextoCadena();
	yybegin(YYINITIAL);
	if (s != null) return s;
}

@\[:\)+\] {
	Symbol s = agregarTextoCadena();
	if (s != null) {
	yypushback(yylength());
	return s;
	}
	return registrarYRetornar(sym.EMOJI_RISA, TipoToken.EMOJI_RISA);
}

@\[:smile:\] {
	Symbol s = agregarTextoCadena();
	if (s != null) {
	yypushback(yylength());
```

```
        return s;
        }
        return registrarYRetornar(sym.EMOJI_RISA, TipoToken.EMOJI_RISA);
}

@\[:\(+\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_TRISTE, TipoToken.EMOJI_TRISTE);
}

@\[:sad:\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_TRISTE, TipoToken.EMOJI_TRISTE);
}

@\[:\|+\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_SERIO, TipoToken.EMOJI_SERIO);
}

@\[:serious:\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_SERIO, TipoToken.EMOJI_SERIO);
}

@\[<+3\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_CORAZON, TipoToken.EMOJI_CORAZON);
```

```
}

@\[:heart:\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_CORAZON, TipoToken.EMOJI_CORAZON);
}

@\[:star:\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_ESTRELLA, TipoToken.EMOJI_ESTRELLA);
}

@\[:star:[0-9]+:\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_ESTRELLA, TipoToken.EMOJI_ESTRELLA);
}

@\[:star-[0-9]+:\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_ESTRELLA, TipoToken.EMOJI_ESTRELLA);
}

@\[:cat:\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_GATO, TipoToken.EMOJI_GATO);
}

@\[:\^\^:\] {
```

```
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_GATO, TipoToken.EMOJI_GATO);
}

[^@\n\r\"]+ {
        if (bufferCadena.length() == 0) {
        lineaCadena = yyline + 1;
        columnaCadena = yycolumn + 1;
        }
        bufferCadena.append(yytext());
}

@ {
        if (bufferCadena.length() == 0) {
        lineaCadena = yyline + 1;
        columnaCadena = yycolumn + 1;
        }
        bufferCadena.append("@");
}

\\\" {
        if (bufferCadena.length() == 0) {
        lineaCadena = yyline + 1;
        columnaCadena = yycolumn + 1;
        }
        bufferCadena.append("\"");
}

\n {
        if (bufferCadena.length() == 0) {
        lineaCadena = yyline + 1;
        columnaCadena = yycolumn + 1;
        }
        bufferCadena.append("\n");
}

<<EOF>> {
        Symbol s = agregarTextoCadena();
        if (s != null) return s;
        errores.add(new Token("Cadena no cerrada", yyline + 1, yycolumn + 1,
TipoToken.ERROR));
        return new Symbol(sym.EOF, yyline, yycolumn);
}
```

```
}

[ \t\r\n]+ { manejarEspacio(); }

{COMENTARIO_LINEA} { }
{COMENTARIO_BLOQUE} { }

{COLORRGB} { return registrarYRetornar(sym.COLOR_RGB, TipoToken.COLOR_RGB,
yytext()); }
{COLORHSL} { return registrarYRetornar(sym.COLOR_HSL, TipoToken.COLOR_HSL,
yytext()); }

"+" { return registrarYRetornar(sym.SUMA, TipoToken.SUMA); }
"-" { return registrarYRetornar(sym.RESTA, TipoToken.RESTA); }
"*" { return registrarYRetornar(sym.MULTIPLICACION, TipoToken.MULTIPLICACION); }
"/" { return registrarYRetornar(sym.DIVISION, TipoToken.DIVISION); }
"^" { return registrarYRetornar(sym.POTENCIACION, TipoToken.POTENCIACION); }
"%" { return registrarYRetornar(sym.MODULO, TipoToken.MODULO); }

"(" { return registrarYRetornar(sym.PARENTESIS_APERTURA,
TipoToken.PARENTESIS_APERTURA); }
")" { return registrarYRetornar(sym.PARENTESIS_CIERRE,
TipoToken.PARENTESIS_CIERRE); }

">=" { return registrarYRetornar(sym.MAYOR_O_IGUAL_QUE,
TipoToken.MAYOR_O_IGUAL_QUE); }
"<=" { return registrarYRetornar(sym.MENOR_O_IGUAL_QUE,
TipoToken.MENOR_O_IGUAL_QUE); }
"==" { return registrarYRetornar(sym.IGUALDAD, TipoToken.IGUALDAD); }
"!=" { return registrarYRetornar(sym.DIFERENTE, TipoToken.DIFERENTE); }
">" { return registrarYRetornar(sym.MAYOR_QUE, TipoToken.MAYOR_QUE); }
"<" { return registrarYRetornar(sym.MENOR_QUE, TipoToken.MENOR_QUE); }

"||" { return registrarYRetornar(sym.OR, TipoToken.OR); }
"&&" { return registrarYRetornar(sym.AND, TipoToken.AND); }
"~" { return registrarYRetornar(sym.NEGACION, TipoToken.NEGACION); }

"number" { return registrarYRetornar(sym.VAR_NUMBER, TipoToken.VAR_NUMBER); }
"string" { return registrarYRetornar(sym.VAR_STRING, TipoToken.VAR_STRING); }
"special" { return registrarYRetornar(sym.VAR_SPECIAL, TipoToken.VAR_SPECIAL); }

"OPEN_QUESTION" { return registrarYRetornar(sym.OPEN_QUESTION,
TipoToken.OPEN_QUESTION); }
"DROP_QUESTION" { return registrarYRetornar(sym.DROP_QUESTION,
TipoToken.DROP_QUESTION); }
"SELECT_QUESTION" { return registrarYRetornar(sym.SELECT_QUESTION,
TipoToken.SELECT_QUESTION); }
```

```
"MULTIPLE_QUESTION" { return registrarYRetornar(sym.MULTIPLE_QUESTION,
TipoToken.MULTIPLE_QUESTION); }

"SECTION" { return registrarYRetornar(sym.SECTION, TipoToken.SECTION); }
"TABLE" { return registrarYRetornar(sym.TABLE, TipoToken.TABLE); }
"TEXT" { return registrarYRetornar(sym.TEXT, TipoToken.TEXT); }

"draw" { return registrarYRetornar(sym.DRAW, TipoToken.DRAW); }
"width" { return registrarYRetornar(sym.WIDTH, TipoToken.WIDTH); }
"height" { return registrarYRetornar(sym.HEIGHT, TipoToken.HEIGHT); }

"pointX" { return registrarYRetornar(sym.POINT_X, TipoToken.POINT_X); }
"pointY" { return registrarYRetornar(sym.POINT_Y, TipoToken.POINT_Y); }

"orientation" { return registrarYRetornar(sym.ORIENTATION, TipoToken.ORIENTATION); }

"VERTICAL" { return registrarYRetornar(sym.VERTICAL, TipoToken.VERTICAL); }
"HORIZONTAL" { return registrarYRetornar(sym.HORIZONTAL, TipoToken.HORIZONTAL); }

"content" { return registrarYRetornar(sym.CONTENT, TipoToken.CONTENT); }
"label" { return registrarYRetornar(sym.LABEL, TipoToken.LABEL); }

"options" { return registrarYRetornar(sym.OPTIONS, TipoToken.OPTIONS); }
"correct" { return registrarYRetornar(sym.CORRECT, TipoToken.CORRECT); }

"elements" { return registrarYRetornar(sym.ELEMENTS, TipoToken.ELEMENTS); }
"styles" { return registrarYRetornar(sym.STYLES, TipoToken.STYLES); }

"RED" { return registrarYRetornar(sym.COLOR_PREDEFINIDO,
TipoToken.COLOR_PREDEFINIDO, yytext()); }
"BLUE" { return registrarYRetornar(sym.COLOR_PREDEFINIDO,
TipoToken.COLOR_PREDEFINIDO, yytext()); }
"GREEN" { return registrarYRetornar(sym.COLOR_PREDEFINIDO,
TipoToken.COLOR_PREDEFINIDO, yytext()); }
"PURPLE" { return registrarYRetornar(sym.COLOR_PREDEFINIDO,
TipoToken.COLOR_PREDEFINIDO, yytext()); }
"SKY" { return registrarYRetornar(sym.COLOR_PREDEFINIDO,
TipoToken.COLOR_PREDEFINIDO, yytext()); }
"BLACK" { return registrarYRetornar(sym.COLOR_PREDEFINIDO,
TipoToken.COLOR_PREDEFINIDO, yytext()); }
"WHITE" { return registrarYRetornar(sym.COLOR_PREDEFINIDO,
TipoToken.COLOR_PREDEFINIDO, yytext()); }

"MONO" { return registrarYRetornar(sym.FUENTE_PREDEFINIDA,
TipoToken.FUENTE_PREDEFINIDA, yytext()); }
"SANS_SERIF" { return registrarYRetornar(sym.FUENTE_PREDEFINIDA,
TipoToken.FUENTE_PREDEFINIDA, yytext()); }
```

```
"CURSIVE" { return registrarYRetornar(sym.FUENTE_PREDEFINIDA,
TipoToken.FUENTE_PREDEFINIDA, yytext()); }

"LINE" { return registrarYRetornar(sym.TIPO_BORDE, TipoToken.TIPO_BORDE, yytext()); }
"DOTTED" { return registrarYRetornar(sym.TIPO_BORDE, TipoToken.TIPO_BORDE,
yytext()); }
"DOUBLE" { return registrarYRetornar(sym.TIPO_BORDE, TipoToken.TIPO_BORDE,
yytext()); }

"IF" { return registrarYRetornar(sym.IF, TipoToken.IF); }
"ELSE" { return registrarYRetornar(sym.ELSE, TipoToken.ELSE); }
"WHILE" { return registrarYRetornar(sym.WHILE, TipoToken.WHILE); }
"DO" { return registrarYRetornar(sym.DO, TipoToken.DO); }
"FOR" { return registrarYRetornar(sym.FOR, TipoToken.FOR); }
"in" { return registrarYRetornar(sym.IN, TipoToken.IN); }

"." { return registrarYRetornar(sym.PUNTO, TipoToken.PUNTO); }

"[" { return registrarYRetornar(sym.CORCHETE_APERTURA,
TipoToken.CORCHETE_APERTURA); }
"]" { return registrarYRetornar(sym.CORCHETE_CIERRE,
TipoToken.CORCHETE_CIERRE); }

"," { return registrarYRetornar(sym.COMA, TipoToken.COMA); }
":" { return registrarYRetornar(sym.DOS_PUNTOS, TipoToken.DOS_PUNTOS); }

"\?" { return registrarYRetornar(sym.COMODIN, TipoToken.COMODIN); }

"=" { return registrarYRetornar(sym.ASIGNACION, TipoToken.ASIGNACION); }

"{" { return registrarYRetornar(sym.LLAVE_APERTURA, TipoToken.LLAVE_APERTURA); }
"}" { return registrarYRetornar(sym.LLAVE_CIERRE, TipoToken.LLAVE_CIERRE); }

";" { return registrarYRetornar(sym.PUNTO_Y_COMA, TipoToken.PUNTO_Y_COMA); }

{DECIMAL} { return registrarYRetornar(sym.DECIMAL, TipoToken.DECIMAL, yytext()); }
{ENTERO} { return registrarYRetornar(sym.ENTERO, TipoToken.ENTERO, yytext()); }

\" {
        bufferCadena.setLength(0);
        lineaCadena = yyline + 1;
        columnaCadena = yycolumn + 1;
        yybegin(STRING);
}

{COLORHEX} { return registrarYRetornar(sym.COLOR_HEXADECIMAL,
TipoToken.COLOR_HEXADECIMAL, yytext()); }
```

```
{IDENT} { return registrarYRetornar(sym.IDENTIFICADOR, TipoToken.IDENTIFICADOR,
yytext()); }

@\[:\)+:\] { return registrarYRetornar(sym.EMOJI_RISA, TipoToken.EMOJI_RISA); }
@\[:\(+:\] { return registrarYRetornar(sym.EMOJI_TRISTE, TipoToken.EMOJI_TRISTE); }
@\[:\|+:\] { return registrarYRetornar(sym.EMOJI_SERIO, TipoToken.EMOJI_SERIO); }
@\[:heart:\] { return registrarYRetornar(sym.EMOJI_CORAZON,
TipoToken.EMOJI_CORAZON); }
@\[:star:\] { return registrarYRetornar(sym.EMOJI_ESTRELLA,
TipoToken.EMOJI_ESTRELLA); }
@\[:star:[0-9]+:\] { return registrarYRetornar(sym.EMOJI_ESTRELLA,
TipoToken.EMOJI_ESTRELLA); }
@\[:cat:\] { return registrarYRetornar(sym.EMOJI_GATO, TipoToken.EMOJI_GATO); }

<<EOF>> { return new Symbol(sym.EOF, yyline, yycolumn); }

. {
	hayError = true;
	errorActual.setLength(0);
	errorActual.append(yytext());
	lineaErrorActual = yyline + 1;
	columnaErrorActual = yycolumn + 1;
	agregarError();
	hayError = false;
	errorActual.setLength(0);
}

package com.mycompany.testlexer2;

import java_cup.runtime.Symbol;
import java.util.ArrayList;
import java.util.List;
import java.io.IOException;

%%

%xstate STRING

%class LexerPKM
%unicode
%line
%column
%public
%cupsym sympkm
%cup
%ignorecase
%extends Lexer
```

```
%{

private List<Token> tokens = new ArrayList<>();
private List<Token> errores = new ArrayList<>();

private boolean hayError = false;
private StringBuilder errorActual = new StringBuilder();
private int lineaErrorActual = 0;
private int columnaErrorActual = 0;

private StringBuilder bufferCadena = new StringBuilder();
private int lineaCadena = 0;
private int columnaCadena = 0;

public List<Token> getTokens(){
	return tokens;
}

public List<Token> getErrores(){
	return errores;
}

private Symbol simbolo(int type){
	return new Symbol(type, yyline, yycolumn);
}

private Symbol simbolo(int type, String value){
	return new Symbol(type, yyline, yycolumn, value);
}

private void agregarToken(TipoToken tipo){
	if(hayError){
	errorActual.append(yytext());
	lineaErrorActual = yyline + 1;
	columnaErrorActual = yycolumn + 1;
	}else{
	tokens.add(new Token(yytext(), yyline + 1, yycolumn + 1, tipo));
	}
}

private void agregarToken(String lexema, int linea, int columna, TipoToken tipo){
	tokens.add(new Token(lexema, linea, columna, tipo));
}

private void agregarError(){
	errores.add(new Token(errorActual.toString(), lineaErrorActual, columnaErrorActual,
TipoToken.ERROR));
}
```

```
private void manejarEspacio(){
        if(hayError && errorActual.length() > 0){
        hayError = false;
        agregarError();
        errorActual.setLength(0);
        }
}

private Symbol agregarTextoCadena(){
        if(bufferCadena.length() > 0){
        String texto = bufferCadena.toString();
        agregarToken(texto, lineaCadena, columnaCadena, TipoToken.CADENA);
        bufferCadena.setLength(0);
        return new Symbol(sym.CADENA, lineaCadena - 1, columnaCadena - 1, texto);
        }
        return null;
}

private Symbol registrarYRetornar(int cupSym, TipoToken tipo){
        agregarToken(tipo);
        return simbolo(cupSym);
}

private Symbol registrarYRetornar(int cupSym, TipoToken tipo, String value){
        agregarToken(tipo);
        return simbolo(cupSym, value);
}

%}

DIGITO = [0-9]
LETRA = [a-zA-Z_]
IDENT = {LETRA}({LETRA}|{DIGITO})*

ENTERO = {DIGITO}+
DECIMAL = {DIGITO}+\.{DIGITO}+

COLORHEX = "#"[0-9a-fA-F]{6}
COLORRGB = rgb\(\s*{ENTERO}\s*,\s*{ENTERO}\s*,\s*{ENTERO}\s*\)

COLORHSL = \<\s*{ENTERO}\s*,\s*{ENTERO}\s*,\s*{ENTERO}\s*\>


COMENTARIO_LINEA = "# "[^\n]*

COMENTARIO_BLOQUE = "###"([^#]|#([^#]|#([^#])))*"###"
```

```
%%

<STRING>{

\" {
        Symbol s = agregarTextoCadena();
        yybegin(YYINITIAL);
        if (s != null) return s;
}

@\[:\)+\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_RISA, TipoToken.EMOJI_RISA);
}

@\[:smile:\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_RISA, TipoToken.EMOJI_RISA);
}

@\[:\(+\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_TRISTE, TipoToken.EMOJI_TRISTE);
}

@\[:sad:\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_TRISTE, TipoToken.EMOJI_TRISTE);
}

@\[:\|+\] {
```

```
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_SERIO, TipoToken.EMOJI_SERIO);
}

@\[:serious:\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_SERIO, TipoToken.EMOJI_SERIO);
}

@\[<+3\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_CORAZON, TipoToken.EMOJI_CORAZON);
}

@\[:heart:\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_CORAZON, TipoToken.EMOJI_CORAZON);
}

@\[:star:\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_ESTRELLA, TipoToken.EMOJI_ESTRELLA);
}

@\[:star:[0-9]+:\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
```

```
        return s;
        }
        return registrarYRetornar(sym.EMOJI_ESTRELLA, TipoToken.EMOJI_ESTRELLA);
}

@\[:star-[0-9]+:\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_ESTRELLA, TipoToken.EMOJI_ESTRELLA);
}

@\[:cat:\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_GATO, TipoToken.EMOJI_GATO);
}

@\[:\^\^:\] {
        Symbol s = agregarTextoCadena();
        if (s != null) {
        yypushback(yylength());
        return s;
        }
        return registrarYRetornar(sym.EMOJI_GATO, TipoToken.EMOJI_GATO);
}

[^@\n\r\"]+ {
        if (bufferCadena.length() == 0) {
        lineaCadena = yyline + 1;
        columnaCadena = yycolumn + 1;
        }
        bufferCadena.append(yytext());
}

@ {
        if (bufferCadena.length() == 0) {
        lineaCadena = yyline + 1;
        columnaCadena = yycolumn + 1;
        }
        bufferCadena.append("@");
}
```

```
\\\" {
        if (bufferCadena.length() == 0) {
        lineaCadena = yyline + 1;
        columnaCadena = yycolumn + 1;
        }
        bufferCadena.append("\"");
}

\n {
        if (bufferCadena.length() == 0) {
        lineaCadena = yyline + 1;
        columnaCadena = yycolumn + 1;
        }
        bufferCadena.append("\n");
}

<<EOF>> {
        Symbol s = agregarTextoCadena();
        if (s != null) return s;
        errores.add(new Token("Cadena no cerrada", yyline + 1, yycolumn + 1,
TipoToken.ERROR));
        return new Symbol(sym.EOF, yyline, yycolumn);
}

}

[ \t\r\n]+ { manejarEspacio(); }

{COLORHEX} { return registrarYRetornar(sym.COLOR_HEXADECIMAL,
TipoToken.COLOR_HEXADECIMAL, yytext()); }
{COLORRGB} { return registrarYRetornar(sym.COLOR_RGB, TipoToken.COLOR_RGB,
yytext()); }
{COLORHSL} { return registrarYRetornar(sym.COLOR_HSL, TipoToken.COLOR_HSL,
yytext()); }

{COMENTARIO_LINEA} { }
{COMENTARIO_BLOQUE} { }

"style" { return registrarYRetornar(sym.STYLE, TipoToken.STYLE); }
"color" { return registrarYRetornar(sym.COLOR, TipoToken.COLOR); }
"background" { return registrarYRetornar(sym.BACKGROUND, TipoToken.BACKGROUND);
}
"font" { return registrarYRetornar(sym.FONT, TipoToken.FONT); }
"family" { return registrarYRetornar(sym.FAMILY, TipoToken.FAMILY); }
"text" { return registrarYRetornar(sym.TEXT, TipoToken.TEXT); }
"size" { return registrarYRetornar(sym.SIZE, TipoToken.SIZE); }
"section" { return registrarYRetornar(sym.SECTION, TipoToken.SECTION); }
"content" { return registrarYRetornar(sym.CONTENT, TipoToken.CONTENT); }
```

```
"table" { return registrarYRetornar(sym.TABLE, TipoToken.TABLE); }
"line" { return registrarYRetornar(sym.LINE, TipoToken.LINE); }
"element" { return registrarYRetornar(sym.ELEMENT, TipoToken.ELEMENT); }
"open" { return registrarYRetornar(sym.OPEN, TipoToken.OPEN); }
"drop" { return registrarYRetornar(sym.DROP, TipoToken.DROP); }
"select"   { return registrarYRetornar(sym.SELECT, TipoToken.SELECT); }
"multiple" { return registrarYRetornar(sym.MULTIPLE, TipoToken.MULTIPLE); }
"border" { return registrarYRetornar(sym.BORDER, TipoToken.BORDER); }



"-" { return registrarYRetornar(sym.RESTA, TipoToken.RESTA); }
"/" { return registrarYRetornar(sym.DIVISION, TipoToken.DIVISION); }

"(" { return registrarYRetornar(sym.PARENTESIS_APERTURA,
TipoToken.PARENTESIS_APERTURA); }
")" { return registrarYRetornar(sym.PARENTESIS_CIERRE,
TipoToken.PARENTESIS_CIERRE); }

">" { return registrarYRetornar(sym.MAYOR_QUE, TipoToken.MAYOR_QUE); }
"<" { return registrarYRetornar(sym.MENOR_QUE, TipoToken.MENOR_QUE); }

"VERTICAL" { return registrarYRetornar(sym.VERTICAL, TipoToken.VERTICAL); }
"HORIZONTAL" { return registrarYRetornar(sym.HORIZONTAL, TipoToken.HORIZONTAL); }

"RED" { return registrarYRetornar(sym.COLOR_PREDEFINIDO,
TipoToken.COLOR_PREDEFINIDO, yytext()); }
"BLUE" { return registrarYRetornar(sym.COLOR_PREDEFINIDO,
TipoToken.COLOR_PREDEFINIDO, yytext()); }
"GREEN" { return registrarYRetornar(sym.COLOR_PREDEFINIDO,
TipoToken.COLOR_PREDEFINIDO, yytext()); }
"PURPLE" { return registrarYRetornar(sym.COLOR_PREDEFINIDO,
TipoToken.COLOR_PREDEFINIDO, yytext()); }
"SKY" { return registrarYRetornar(sym.COLOR_PREDEFINIDO,
TipoToken.COLOR_PREDEFINIDO, yytext()); }
"BLACK" { return registrarYRetornar(sym.COLOR_PREDEFINIDO,
TipoToken.COLOR_PREDEFINIDO, yytext()); }
"WHITE" { return registrarYRetornar(sym.COLOR_PREDEFINIDO,
TipoToken.COLOR_PREDEFINIDO, yytext()); }

"MONO" { return registrarYRetornar(sym.FUENTE_PREDEFINIDA,
TipoToken.FUENTE_PREDEFINIDA, yytext()); }
"SANS_SERIF" { return registrarYRetornar(sym.FUENTE_PREDEFINIDA,
TipoToken.FUENTE_PREDEFINIDA, yytext()); }
"CURSIVE" { return registrarYRetornar(sym.FUENTE_PREDEFINIDA,
TipoToken.FUENTE_PREDEFINIDA, yytext()); }
```

```
"DOTTED" { return registrarYRetornar(sym.TIPO_BORDE, TipoToken.TIPO_BORDE,
yytext()); }
"DOUBLE" { return registrarYRetornar(sym.TIPO_BORDE, TipoToken.TIPO_BORDE,
yytext()); }

"," { return registrarYRetornar(sym.COMA, TipoToken.COMA); }

"=" { return registrarYRetornar(sym.ASIGNACION, TipoToken.ASIGNACION); }

"{" { return registrarYRetornar(sym.LLAVE_APERTURA, TipoToken.LLAVE_APERTURA); }
"}" { return registrarYRetornar(sym.LLAVE_CIERRE, TipoToken.LLAVE_CIERRE); }


{ENTERO} { return registrarYRetornar(sym.ENTERO, TipoToken.ENTERO, yytext()); }

\" {
	bufferCadena.setLength(0);
	lineaCadena = yyline + 1;
	columnaCadena = yycolumn + 1;
	yybegin(STRING);
}



@\[:\)+:\] { return registrarYRetornar(sym.EMOJI_RISA, TipoToken.EMOJI_RISA); }
@\[:\(+:\] { return registrarYRetornar(sym.EMOJI_TRISTE, TipoToken.EMOJI_TRISTE); }
@\[:\|+:\] { return registrarYRetornar(sym.EMOJI_SERIO, TipoToken.EMOJI_SERIO); }
@\[:heart:\] { return registrarYRetornar(sym.EMOJI_CORAZON,
TipoToken.EMOJI_CORAZON); }
@\[:star:\] { return registrarYRetornar(sym.EMOJI_ESTRELLA,
TipoToken.EMOJI_ESTRELLA); }
@\[:star:[0-9]+:\] { return registrarYRetornar(sym.EMOJI_ESTRELLA,
TipoToken.EMOJI_ESTRELLA); }
@\[:cat:\] { return registrarYRetornar(sym.EMOJI_GATO, TipoToken.EMOJI_GATO); }

<<EOF>> { return new Symbol(sym.EOF, yyline, yycolumn); }

. {
	hayError = true;
	errorActual.setLength(0);
	errorActual.append(yytext());
	lineaErrorActual = yyline + 1;
	columnaErrorActual = yycolumn + 1;
	agregarError();
	hayError = false;
	errorActual.setLength(0);
}
```

Archivos de configuración cup

```
package com.mycompany.testlexer2;

import java_cup.runtime.*;
import java.util.ArrayList;
import java.util.List;

import com.mycompany.testlexer2.ast.*;

parser code {:
	private final List<String> erroresSintacticos = new ArrayList<>();
	public Symbol ultimoError = null;

	public List<String> getErroresSintacticos(){
	return erroresSintacticos;
	}

	private int linea(Symbol s) {
	return s == null ? -1 : s.left + 1;
	}

	private int columna(Symbol s) {
	return s == null ? -1 : s.right + 1;
	}

	private String ubicacion(Symbol s) {
	if (s == null) return "fin de entrada";
	return "línea " + linea(s) + ", columna " + columna(s);
	}

	@Override
	public void report_error(String message, Object info) {
	if (info instanceof Symbol s) {
	ultimoError = s;
	erroresSintacticos.add(
		"Error sintáctico en " + ubicacion(s) + ": " + message
	);
	} else {
	erroresSintacticos.add("Error sintáctico: " + message);
	}
	}

	@Override
	public void syntax_error(Symbol cur_token) {
	ultimoError = cur_token;
	if (cur_token != null) {
```

```
	erroresSintacticos.add(
		"Token inesperado en " + ubicacion(cur_token)
	);
	} else {
	erroresSintacticos.add("Error sintáctico: fin de entrada inesperado");
	}
	}

	@Override
	public void unrecovered_syntax_error(Symbol cur_token) throws Exception {
	ultimoError = cur_token;
	if (cur_token != null) {
	erroresSintacticos.add(
		"No se pudo recuperar del error en " + ubicacion(cur_token)
	);
	} else {
	erroresSintacticos.add(
		"No se pudo recuperar del error al final de la entrada"
	);
	}
	throw new Exception("Error sintáctico: " + (ultimoError.left + 1) + ":" +
(ultimoError.right + 1));
	}
:};

scan with {:
	return getScanner().next_token();
:};

terminal String IDENTIFICADOR;
terminal String ENTERO;
terminal String DECIMAL;
terminal String CADENA;
terminal String COLOR_HEXADECIMAL;
terminal String COLOR_RGB;
terminal String COLOR_HSL;

terminal SUMA, RESTA, MULTIPLICACION, DIVISION, POTENCIACION, MODULO;

terminal PARENTESIS_APERTURA, PARENTESIS_CIERRE;
terminal CORCHETE_APERTURA, CORCHETE_CIERRE;
terminal LLAVE_APERTURA, LLAVE_CIERRE;

terminal MAYOR_QUE, MENOR_QUE, MAYOR_O_IGUAL_QUE,
MENOR_O_IGUAL_QUE, IGUALDAD, DIFERENTE;

terminal OR, AND, NEGACION;
```

```
terminal VAR_NUMBER, VAR_STRING, VAR_SPECIAL;

terminal OPEN_QUESTION, DROP_QUESTION, SELECT_QUESTION,
MULTIPLE_QUESTION;
terminal SECTION, TABLE, TEXT;

terminal DRAW, WIDTH, HEIGHT, POINT_X, POINT_Y, ORIENTATION;
terminal CONTENT, LABEL, OPTIONS, CORRECT, ELEMENTS, STYLES;

terminal VERTICAL, HORIZONTAL;
terminal String COLOR_PREDEFINIDO;
terminal String FUENTE_PREDEFINIDA;
terminal String TIPO_BORDE;

terminal IF, ELSE, WHILE, DO, FOR, IN;

terminal PUNTO, COMA, DOS_PUNTOS, COMODIN, ASIGNACION, PUNTO_Y_COMA;

terminal EMOJI_RISA, EMOJI_TRISTE, EMOJI_SERIO, EMOJI_CORAZON,
EMOJI_ESTRELLA, EMOJI_GATO;

terminal UMINUS;

non terminal NodoPrograma programa;
non terminal List<NodoSentencia> lista_sentencias;
non terminal NodoSentencia sentencia;
non terminal NodoSentencia sentencia_superior;
non terminal NodoBloque bloque;
non terminal List<NodoSentencia> lista_sentencias_bloque;
non terminal NodoSentencia sentencia_bloque;

non terminal NodoSentencia sentencia_declaracion_variable;
non terminal NodoSentencia sentencia_asignacion;
non terminal NodoSentencia sentencia_draw;
non terminal NodoSentencia sentencia_elemento;
non terminal NodoSentencia sentencia_control;

non terminal NodoDeclaracionVariable declaracion_variable_simple;
non terminal NodoDeclaracionSpecial declaracion_variable_special;

non terminal NodoLlamadaMetodo llamada_draw;
non terminal List<NodoExpresion> lista_argumentos_opcional;
non terminal List<NodoExpresion> lista_argumentos;
non terminal List<NodoExpresion> lista_argumentos_resto;


non terminal NodoElemento elemento;
non terminal NodoElemento elemento_seccion;
```

```
non terminal NodoElemento elemento_tabla;
non terminal NodoElemento elemento_texto;
non terminal NodoPregunta elemento_pregunta;

non terminal List<NodoAtributo> lista_atributos_seccion;
non terminal List<NodoAtributo> lista_atributos_tabla;
non terminal List<NodoAtributo> lista_atributos_texto;
non terminal List<NodoAtributo> lista_atributos_pregunta;
non terminal NodoAtributo atributo_seccion;
non terminal NodoAtributo atributo_tabla;
non terminal NodoAtributo atributo_texto;
non terminal NodoAtributo atributo_pregunta;

non terminal NodoAtributo atributo_bloque_estilos;
non terminal NodoEstilo entrada_estilo;
non terminal List<NodoEstilo> lista_estilos_no_vacia;

non terminal List<NodoElemento> lista_elementos_opcional;
non terminal List<NodoElemento> lista_elementos;

non terminal List<List<NodoElemento>> filas_tabla;
non terminal List<NodoElemento> fila_tabla;
non terminal NodoElemento celda_tabla;
non terminal List<NodoElemento> lista_celdas_tabla;

non terminal NodoExpresion expresion;
non terminal NodoExpresion cadena_or;
non terminal NodoExpresion cadena_and;
non terminal NodoExpresion expresion_comparacion;
non terminal NodoExpresion expresion_aritmetica;
non terminal NodoExpresion termino;
non terminal NodoExpresion expresion_potencia;
non terminal NodoExpresion expresion_unaria;
non terminal NodoExpresion expresion_primaria;
non terminal NodoExpresion literal;
non terminal NodoExpresion referencia_identificador;
non terminal NodoExpresion literal_cadena;
non terminal List<NodoExpresion> partes_cadena;
non terminal NodoExpresion parte_cadena;

non terminal List<NodoExpresion> lista_opciones_no_vacia;
non terminal List<NodoExpresion> lista_expresiones;
non terminal List<NodoExpresion> lista_expresiones_opcional;

non terminal NodoIf sentencia_if;
non terminal NodoColaIf cola_if;

non terminal NodoWhile sentencia_while;
```

```
non terminal NodoDoWhile sentencia_do_while;
non terminal NodoSentencia sentencia_for;
non terminal NodoForClasico sentencia_for_clasico;
non terminal NodoForRango sentencia_for_rango;

non terminal NodoAsignacion inicializacion_for;
non terminal NodoExpresion condicion_for;
non terminal NodoAsignacion actualizacion_for;

non terminal String nombre_tipo;

precedence left OR;
precedence left AND;
precedence right NEGACION;
precedence nonassoc MAYOR_QUE, MENOR_QUE, MAYOR_O_IGUAL_QUE,
MENOR_O_IGUAL_QUE, IGUALDAD, DIFERENTE;
precedence left SUMA, RESTA;
precedence left MULTIPLICACION, DIVISION;
precedence left POTENCIACION, MODULO;
precedence right UMINUS;

start with programa;

programa ::=
        lista_sentencias:ls
        {: RESULT = new NodoPrograma(ls, lsleft, lsright); :}
        ;

lista_sentencias ::=
        lista_sentencias:ls sentencia:s
        {: ls.add(s); RESULT = ls; :}
        | sentencia:s
        {:
        List<NodoSentencia> lista = new ArrayList<>();
        lista.add(s);
        RESULT = lista;
        :}
        ;

sentencia ::=
        sentencia_superior:s
        {: RESULT = s; :}
        | sentencia_elemento:s
        {: RESULT = s; :}
        | sentencia_control:s
        {: RESULT = s; :}
        ;
```

```
sentencia_superior ::=
        sentencia_declaracion_variable:s
        {: RESULT = s; :}
        | sentencia_asignacion:s
        {: RESULT = s; :}
        | sentencia_draw:s
        {: RESULT = s; :}
        ;

bloque ::=
        LLAVE_APERTURA:la lista_sentencias_bloque:ls LLAVE_CIERRE:lc
        {: RESULT = new NodoBloque(ls, laleft, lcright); :}
        | LLAVE_APERTURA:la error LLAVE_CIERRE:lc
        {:
        erroresSintacticos.add(
        "Se recuperó un bloque con errores en " + ubicacion(ultimoError)
        );
        RESULT = new NodoBloque(new ArrayList<NodoSentencia>(), laleft, lcright);
        :}
        ;

lista_sentencias_bloque ::=
        lista_sentencias_bloque:ls sentencia_bloque:s
        {: ls.add(s); RESULT = ls; :}
        |
        {: RESULT = new ArrayList<NodoSentencia>(); :}
        ;

sentencia_bloque ::=
        sentencia_declaracion_variable:s
        {: RESULT = s; :}
        | sentencia_asignacion:s
        {: RESULT = s; :}
        | sentencia_draw:s
        {: RESULT = s; :}
        | sentencia_elemento:s
        {: RESULT = s; :}
        | sentencia_control:s
        {: RESULT = s; :}
        ;

declaracion_variable_simple ::=
        nombre_tipo:t IDENTIFICADOR:id
        {: RESULT = new NodoDeclaracionVariable(t, id, null, tleft, idright); :}
        | nombre_tipo:t IDENTIFICADOR:id ASIGNACION:a expresion:e
        {: RESULT = new NodoDeclaracionVariable(t, id, e, tleft, eright); :}
        ;
```

```
sentencia_declaracion_variable ::=
        declaracion_variable_simple:d
        {: RESULT = d; :}
        | declaracion_variable_special:d
        {: RESULT = d; :}
        ;

declaracion_variable_special ::=
        VAR_SPECIAL:s IDENTIFICADOR:id ASIGNACION:a elemento_pregunta:p
        {: RESULT = new NodoDeclaracionSpecial(id, p, sleft, pright); :}
        ;

nombre_tipo ::=
        VAR_NUMBER
        {: RESULT = "number"; :}
        | VAR_STRING
        {: RESULT = "string"; :}
        ;

sentencia_asignacion ::=
        IDENTIFICADOR:id ASIGNACION:a expresion:e
        {: RESULT = new NodoAsignacion(id, e, idleft, eright); :}
        ;

sentencia_draw ::=
        llamada_draw:ld
        {: RESULT = ld; :}
        ;

llamada_draw ::=
        IDENTIFICADOR:id PUNTO:p DRAW:d PARENTESIS_APERTURA:pa
lista_argumentos_opcional:args PARENTESIS_CIERRE:pc
        {: RESULT = new NodoLlamadaMetodo(id, "draw", args, idleft, pcright); :}
        ;

lista_argumentos_opcional ::=
        lista_argumentos:la
        {: RESULT = la; :}
        |
        {: RESULT = new ArrayList<NodoExpresion>(); :}
        ;

lista_argumentos ::=
        expresion:e lista_argumentos_resto:resto
        {:
        List<NodoExpresion> lista = new ArrayList<>();
        lista.add(e);
        lista.addAll(resto);
```

```
        RESULT = lista;
        :}
        ;

lista_argumentos_resto ::=
        COMA expresion:e lista_argumentos_resto:resto
        {:
        List<NodoExpresion> lista = new ArrayList<>();
        lista.add(e);
        lista.addAll(resto);
        RESULT = lista;
        :}
        |
        {: RESULT = new ArrayList<NodoExpresion>(); :}
        ;

sentencia_elemento ::=
        elemento:e
        {: RESULT = new NodoSentenciaElemento(e, eleft, eright); :}
        ;

elemento ::=
        elemento_seccion:e
        {: RESULT = e; :}
        | elemento_tabla:e
        {: RESULT = e; :}
        | elemento_texto:e
        {: RESULT = e; :}
        | elemento_pregunta:e
        {: RESULT = e; :}
        ;

elemento_seccion ::=
        SECTION:s CORCHETE_APERTURA:ca lista_atributos_seccion:attrs
CORCHETE_CIERRE:cc
        {: RESULT = new NodoSeccion(attrs, sleft, ccright); :}
        ;

lista_atributos_seccion ::=
        lista_atributos_seccion:la COMA:c atributo_seccion:a
        {: la.add(a); RESULT = la; :}
        | atributo_seccion:a
        {:
        List<NodoAtributo> lista = new ArrayList<>();
        lista.add(a);
        RESULT = lista;
        :}
        ;
```

```
atributo_seccion ::=
        WIDTH:w DOS_PUNTOS:d expresion:e
        {: RESULT = new NodoAtributo("width", e, wleft, eright); :}
        | HEIGHT:h DOS_PUNTOS:d expresion:e
        {: RESULT = new NodoAtributo("height", e, hleft, eright); :}
        | POINT_X:px DOS_PUNTOS:d expresion:e
        {: RESULT = new NodoAtributo("pointX", e, pxleft, eright); :}
        | POINT_Y:py DOS_PUNTOS:d expresion:e
        {: RESULT = new NodoAtributo("pointY", e, pyleft, eright); :}
        | ORIENTATION:o DOS_PUNTOS:d expresion:e
        {: RESULT = new NodoAtributo("orientation", e, oleft, eright); :}
        | ELEMENTS:el DOS_PUNTOS:d LLAVE_APERTURA:la
lista_elementos_opcional:els LLAVE_CIERRE:lc
        {: RESULT = new NodoAtributo("elements", new NodoListaElementos(els, laleft,
lcright), elleft, lcright); :}
        | STYLES:st atributo_bloque_estilos:abe
        {: RESULT = new NodoAtributo("styles", abe.getValor(), stleft, aberight); :}
        ;

elemento_tabla ::=
        TABLE:t CORCHETE_APERTURA:ca lista_atributos_tabla:attrs
CORCHETE_CIERRE:cc
        {: RESULT = new NodoTabla(attrs, tleft, ccright); :}
        ;

lista_atributos_tabla ::=
        lista_atributos_tabla:la COMA:c atributo_tabla:a
        {: la.add(a); RESULT = la; :}
        | atributo_tabla:a
        {:
        List<NodoAtributo> lista = new ArrayList<>();
        lista.add(a);
        RESULT = lista;
        :}
        ;

atributo_tabla ::=
        WIDTH:w DOS_PUNTOS:d expresion:e
        {: RESULT = new NodoAtributo("width", e, wleft, eright); :}
        | HEIGHT:h DOS_PUNTOS:d expresion:e
        {: RESULT = new NodoAtributo("height", e, hleft, eright); :}
        | POINT_X:px DOS_PUNTOS:d expresion:e
        {: RESULT = new NodoAtributo("pointX", e, pxleft, eright); :}
        | POINT_Y:py DOS_PUNTOS:d expresion:e
        {: RESULT = new NodoAtributo("pointY", e, pyleft, eright); :}
        | ELEMENTS:el DOS_PUNTOS:d LLAVE_APERTURA:la filas_tabla:ft
LLAVE_CIERRE:lc
```

```
        {: RESULT = new NodoAtributo("elements", new NodoFilasTabla(ft, laleft, lcright),
elleft, lcright); :}
        | STYLES:st atributo_bloque_estilos:abe
        {: RESULT = new NodoAtributo("styles", abe.getValor(), stleft, aberight); :}
        ;

elemento_texto ::=
        TEXT:t CORCHETE_APERTURA:ca lista_atributos_texto:attrs
CORCHETE_CIERRE:cc
        {: RESULT = new NodoTexto(attrs, tleft, ccright); :}
        ;

lista_atributos_texto ::=
        lista_atributos_texto:la COMA:c atributo_texto:a
        {: la.add(a); RESULT = la; :}
        | atributo_texto:a
        {:
        List<NodoAtributo> lista = new ArrayList<>();
        lista.add(a);
        RESULT = lista;
        :}
        ;

atributo_texto ::=
        WIDTH:w DOS_PUNTOS:d expresion:e
        {: RESULT = new NodoAtributo("width", e, wleft, eright); :}
        | HEIGHT:h DOS_PUNTOS:d expresion:e
        {: RESULT = new NodoAtributo("height", e, hleft, eright); :}
        | CONTENT:c DOS_PUNTOS:d expresion:e
        {: RESULT = new NodoAtributo("content", e, cleft, eright); :}
        | STYLES:st atributo_bloque_estilos:abe
        {: RESULT = new NodoAtributo("styles", abe.getValor(), stleft, aberight); :}
        ;

elemento_pregunta ::=
        OPEN_QUESTION:o CORCHETE_APERTURA:ca lista_atributos_pregunta:attrs
CORCHETE_CIERRE:cc
        {: RESULT = new NodoPreguntaAbierta(attrs, oleft, ccright); :}
        | DROP_QUESTION:o CORCHETE_APERTURA:ca lista_atributos_pregunta:attrs
CORCHETE_CIERRE:cc
        {: RESULT = new NodoPreguntaDesplegable(attrs, oleft, ccright); :}
        | SELECT_QUESTION:o CORCHETE_APERTURA:ca lista_atributos_pregunta:attrs
CORCHETE_CIERRE:cc
        {: RESULT = new NodoPreguntaSeleccionUnica(attrs, oleft, ccright); :}
        | MULTIPLE_QUESTION:o CORCHETE_APERTURA:ca
lista_atributos_pregunta:attrs CORCHETE_CIERRE:cc
        {: RESULT = new NodoPreguntaSeleccionMultiple(attrs, oleft, ccright); :}
        ;
```

```
lista_atributos_pregunta ::=
        lista_atributos_pregunta:la COMA:c atributo_pregunta:a
        {: la.add(a); RESULT = la; :}
        | atributo_pregunta:a
        {:
        List<NodoAtributo> lista = new ArrayList<>();
        lista.add(a);
        RESULT = lista;
        :}
        ;

atributo_pregunta ::=
        WIDTH:w DOS_PUNTOS:d expresion:e
        {: RESULT = new NodoAtributo("width", e, wleft, eright); :}
        | HEIGHT:h DOS_PUNTOS:d expresion:e
        {: RESULT = new NodoAtributo("height", e, hleft, eright); :}
        | LABEL:l DOS_PUNTOS:d expresion:e
        {: RESULT = new NodoAtributo("label", e, lleft, eright); :}
        | OPTIONS:o DOS_PUNTOS:d LLAVE_APERTURA:la lista_opciones_no_vacia:opts
LLAVE_CIERRE:lc
        {: RESULT = new NodoAtributo("options", new NodoListaExpresiones(opts, laleft,
lcright), oleft, lcright); :}
        | OPTIONS:o DOS_PUNTOS:d IDENTIFICADOR PARENTESIS_APERTURA:pa
IDENTIFICADOR COMA ENTERO:ri COMA ENTERO:rs PARENTESIS_CIERRE:pc
        {:
        List<NodoExpresion> opts = new ArrayList<>();
        NodoLiteral l1 =   new NodoLiteral("ri", ri, 1, 1);
        NodoLiteral l2 =   new NodoLiteral("rs", rs, 1, 1);
        opts.add(l1);
        opts.add(l2);
        RESULT = new NodoAtributo("options", new NodoListaExpresiones(opts, paleft,
pcright), oleft, pcright); :}
        | CORRECT:c DOS_PUNTOS:d expresion:e
        {: RESULT = new NodoAtributo("correct", e, cleft, eright); :}
        | CORRECT:c DOS_PUNTOS:d LLAVE_APERTURA:la lista_expresiones:lex
LLAVE_CIERRE:lc
        {: RESULT = new NodoAtributo("correct", new NodoListaExpresiones(lex, laleft,
lcright), cleft, lcright); :}
        | STYLES:st atributo_bloque_estilos:abe
        {: RESULT = new NodoAtributo("styles", abe.getValor(), stleft, aberight); :}
        ;

atributo_bloque_estilos ::=
        CORCHETE_APERTURA:ca lista_estilos_no_vacia:le CORCHETE_CIERRE:cc
        {: RESULT = new NodoAtributo("styles", new NodoEstilos(le, caleft, ccright), caleft,
ccright); :}
        ;
```

```
lista_estilos_no_vacia ::=
        lista_estilos_no_vacia:le COMA:c entrada_estilo:ee
        {: le.add(ee); RESULT = le; :}
        | entrada_estilo:ee
        {:
        List<NodoEstilo> lista = new ArrayList<>();
        lista.add(ee);
        RESULT = lista;
        :}
        ;

entrada_estilo ::=
        CADENA:key DOS_PUNTOS:d expresion:e
        {: RESULT = new NodoEstilo(key, e, keyleft, eright); :}

        |

        CADENA:key DOS_PUNTOS:d PARENTESIS_APERTURA:pa literal:l COMA:c2
literal:ll COMA:c3 literal:lll PARENTESIS_CIERRE:pc
        {: RESULT = new NodoBorde(key, l, ll, lll, keyleft, lllright); :}
        ;

lista_elementos_opcional ::=
        lista_elementos:le
        {: RESULT = le; :}
        |
        {: RESULT = new ArrayList<NodoElemento>(); :}
        ;

lista_elementos ::=
        lista_elementos:le COMA:c elemento:e
        {: le.add(e); RESULT = le; :}
        | lista_elementos:le COMA:c error
        {:
        erroresSintacticos.add(
        "Elemento inválido en lista en " + ubicacion(ultimoError)
        );
        RESULT = le;
        :}
        | elemento:e
        {:
        List<NodoElemento> lista = new ArrayList<>();
        lista.add(e);
        RESULT = lista;
        :}
        ;
```

```
filas_tabla ::=
        filas_tabla:ft COMA:c fila_tabla:f
        {: ft.add(f); RESULT = ft; :}
        | filas_tabla:ft COMA:c error
        {:
        erroresSintacticos.add(
        "Fila inválida en tabla en " + ubicacion(ultimoError)
        );
        RESULT = ft;
        :}
        | fila_tabla:f
        {:
        List<List<NodoElemento>> lista = new ArrayList<>();
        lista.add(f);
        RESULT = lista;
        :}
        ;

fila_tabla ::=
        CORCHETE_APERTURA:ca lista_celdas_tabla:lct CORCHETE_CIERRE:cc
        {: RESULT = lct; :}
        ;

lista_celdas_tabla ::=
        lista_celdas_tabla:lct COMA:c celda_tabla:ct
        {: lct.add(ct); RESULT = lct; :}
        | lista_celdas_tabla:lct COMA:c error
        {:
        erroresSintacticos.add(
        "Celda inválida en fila de tabla en " + ubicacion(ultimoError)
        );
        RESULT = lct;
        :}
        | celda_tabla:ct
        {:
        List<NodoElemento> lista = new ArrayList<>();
        lista.add(ct);
        RESULT = lista;
        :}
        ;

celda_tabla ::=
        LLAVE_APERTURA:la elemento:e LLAVE_CIERRE:lc
        {: RESULT = e; :}
        ;

expresion ::=
        expresion_comparacion:e
```

```
        {: RESULT = e; :}
        | expresion_comparacion:l OR:op expresion_comparacion:r
        {: RESULT = new NodoExpresionBinaria("||", l, r, lleft, rright); :}
        | cadena_or:l OR:op expresion_comparacion:r
        {: RESULT = new NodoExpresionBinaria("||", l, r, lleft, rright); :}
        | expresion_comparacion:l AND:op expresion_comparacion:r
        {: RESULT = new NodoExpresionBinaria("&&", l, r, lleft, rright); :}
        | cadena_and:l AND:op expresion_comparacion:r
        {: RESULT = new NodoExpresionBinaria("&&", l, r, lleft, rright); :}
        ;

cadena_or ::=
        expresion_comparacion:l OR:op expresion_comparacion:r
        {: RESULT = new NodoExpresionBinaria("||", l, r, lleft, rright); :}
        | cadena_or:l OR:op expresion_comparacion:r
        {: RESULT = new NodoExpresionBinaria("||", l, r, lleft, rright); :}
        ;

cadena_and ::=
        expresion_comparacion:l AND:op expresion_comparacion:r
        {: RESULT = new NodoExpresionBinaria("&&", l, r, lleft, rright); :}
        | cadena_and:l AND:op expresion_comparacion:r
        {: RESULT = new NodoExpresionBinaria("&&", l, r, lleft, rright); :}
        ;

expresion_comparacion ::=
        expresion_comparacion:l MAYOR_QUE:op expresion_aritmetica:r
        {: RESULT = new NodoExpresionBinaria(">", l, r, lleft, rright); :}
        | expresion_comparacion:l MENOR_QUE:op expresion_aritmetica:r
        {: RESULT = new NodoExpresionBinaria("<", l, r, lleft, rright); :}
        | expresion_comparacion:l MAYOR_O_IGUAL_QUE:op expresion_aritmetica:r
        {: RESULT = new NodoExpresionBinaria(">=", l, r, lleft, rright); :}
        | expresion_comparacion:l MENOR_O_IGUAL_QUE:op expresion_aritmetica:r
        {: RESULT = new NodoExpresionBinaria("<=", l, r, lleft, rright); :}
        | expresion_comparacion:l IGUALDAD:op expresion_aritmetica:r
        {: RESULT = new NodoExpresionBinaria("==", l, r, lleft, rright); :}
        | expresion_comparacion:l DIFERENTE:op expresion_aritmetica:r
        {: RESULT = new NodoExpresionBinaria("!!", l, r, lleft, rright); :}
        | expresion_aritmetica:e
        {: RESULT = e; :}
        ;

expresion_aritmetica ::=
        expresion_aritmetica:l SUMA:op termino:r
        {: RESULT = new NodoExpresionBinaria("+", l, r, lleft, rright); :}
        | expresion_aritmetica:l RESTA:op termino:r
        {: RESULT = new NodoExpresionBinaria("-", l, r, lleft, rright); :}
        | termino:e
```

```
        {: RESULT = e; :}
        ;

termino ::=
        termino:l MULTIPLICACION:op expresion_potencia:r
        {: RESULT = new NodoExpresionBinaria("*", l, r, lleft, rright); :}
        | termino:l DIVISION:op expresion_potencia:r
        {: RESULT = new NodoExpresionBinaria("/", l, r, lleft, rright); :}
        | termino:l MODULO:op expresion_potencia:r
        {: RESULT = new NodoExpresionBinaria("%", l, r, lleft, rright); :}
        | expresion_potencia:e
        {: RESULT = e; :}
        ;

expresion_potencia ::=
        expresion_potencia:l POTENCIACION:op expresion_unaria:r
        {: RESULT = new NodoExpresionBinaria("^", l, r, lleft, rright); :}
        | expresion_unaria:e
        {: RESULT = e; :}
        ;

expresion_unaria ::=
        RESTA:op expresion_unaria:e
        {: RESULT = new NodoExpresionUnaria("-", e, opleft, eright); :}
        %prec UMINUS
        | NEGACION:op expresion_unaria:e
        {: RESULT = new NodoExpresionUnaria("~", e, opleft, eright); :}
        | expresion_primaria:e
        {: RESULT = e; :}
        ;

expresion_primaria ::=
        literal:l
        {: RESULT = l; :}
        | referencia_identificador:r
        {: RESULT = r; :}
        | COMODIN:c
        {: RESULT = new NodoComodin(cleft, cright); :}
        | PARENTESIS_APERTURA:pa expresion:e PARENTESIS_CIERRE:pc
        {: RESULT = e; :}
        ;

referencia_identificador ::=
        IDENTIFICADOR:id
        {: RESULT = new NodoIdentificador(id, idleft, idright); :}
        ;

literal ::=
```

```
        literal_cadena:c
        {: RESULT = c; :}
        | COLOR_HEXADECIMAL:v
        {: RESULT = new NodoLiteral("color_hex", v, vleft, vright); :}
        | COLOR_RGB:v
        {: RESULT = new NodoLiteral("color_rgb", v.substring(3), vleft, vright); :}
        | COLOR_HSL:v
        {: RESULT = new NodoLiteral("color_hsl", v, vleft, vright); :}
        | COLOR_PREDEFINIDO:v
        {: RESULT = new NodoLiteral("color_named", v, vleft, vright); :}
        | FUENTE_PREDEFINIDA:v
        {: RESULT = new NodoLiteral("font", v, vleft, vright); :}
        | TIPO_BORDE:v
        {: RESULT = new NodoLiteral("border_type", v, vleft, vright); :}
        | VERTICAL:v
        {: RESULT = new NodoLiteral("orientation", "VERTICAL", vleft, vright); :}
        | HORIZONTAL:h
        {: RESULT = new NodoLiteral("orientation", "HORIZONTAL", hleft, hright); :}
        | ENTERO:v
        {: RESULT = new NodoLiteral("number", v, vleft, vright); :}
        | DECIMAL:v
        {: RESULT = new NodoLiteral("decimal", v, vleft, vright); :}
        ;

literal_cadena ::=
        partes_cadena:pc
        {: RESULT = new NodoCadenaCompuesta(pc, pcleft, pcright); :}
        ;

partes_cadena ::=
        partes_cadena:pc parte_cadena:p
        {: pc.add(p); RESULT = pc; :}
        | parte_cadena:p
        {:
        List<NodoExpresion> lista = new ArrayList<>();
        lista.add(p);
        RESULT = lista;
        :}
        ;

parte_cadena ::=
        CADENA:v
        {: RESULT = new NodoLiteral("string", v, vleft, vright); :}
        | EMOJI_RISA:e
        {: RESULT = new NodoEmoji("risa", eleft, eright); :}
        | EMOJI_TRISTE:e
        {: RESULT = new NodoEmoji("triste", eleft, eright); :}
        | EMOJI_SERIO:e
```

```
        {: RESULT = new NodoEmoji("serio", eleft, eright); :}
        | EMOJI_CORAZON:e
        {: RESULT = new NodoEmoji("corazon", eleft, eright); :}
        | EMOJI_ESTRELLA:e
        {: RESULT = new NodoEmoji("estrella", eleft, eright); :}
        | EMOJI_GATO:e
        {: RESULT = new NodoEmoji("gato", eleft, eright); :}
        ;

lista_opciones_no_vacia ::=
        lista_opciones_no_vacia:lo COMA:c expresion:e
        {: lo.add(e); RESULT = lo; :}
        | lista_opciones_no_vacia:lo COMA:c error
        {:
        erroresSintacticos.add(
        "Opción inválida en " + ubicacion(ultimoError)
        );
        RESULT = lo;
        :}
        | expresion:e
        {:
        List<NodoExpresion> lista = new ArrayList<>();
        lista.add(e);
        RESULT = lista;
        :}
        ;

lista_expresiones ::=
        lista_expresiones:le COMA:c expresion:e
        {: le.add(e); RESULT = le; :}
        | lista_expresiones:le COMA:c error
        {:
        erroresSintacticos.add(
        "Expresión inválida en lista en " + ubicacion(ultimoError)
        );
        RESULT = le;
        :}
        | expresion:e
        {:
        List<NodoExpresion> lista = new ArrayList<>();
        lista.add(e);
        RESULT = lista;
        :}
        ;

lista_expresiones_opcional ::=
        lista_expresiones:le
        {: RESULT = le; :}
```

```
        |
        {: RESULT = new ArrayList<NodoExpresion>(); :}
        ;

sentencia_control ::=
        sentencia_if:s
        {: RESULT = s; :}
        | sentencia_while:s
        {: RESULT = s; :}
        | sentencia_do_while:s
        {: RESULT = s; :}
        | sentencia_for:s
        {: RESULT = s; :}
        ;

sentencia_if ::=
        IF:tokif PARENTESIS_APERTURA:pa expresion:cond PARENTESIS_CIERRE:pc
bloque:b cola_if:c
        {: RESULT = new NodoIf(cond, b, c, tokifleft, cright); :}
        | IF:tokif PARENTESIS_APERTURA:pa expresion:cond PARENTESIS_CIERRE:pc
error bloque:b cola_if:c
        {:
        erroresSintacticos.add(
        "Se ignoró contenido inválido entre la condición del IF y el bloque en " +
        ubicacion(ultimoError)
        );
        RESULT = new NodoIf(cond, b, c, tokifleft, cright);
        :}
        ;

cola_if ::=
        ELSE:tokelse IF:tokif PARENTESIS_APERTURA:pa expresion:cond
PARENTESIS_CIERRE:pc bloque:b cola_if:c
        {: RESULT = new NodoElseIf(cond, b, c, tokelseleft, cright); :}
        | ELSE:tokelse error bloque:b
        {:
        erroresSintacticos.add(
        "Se ignoró contenido inválido entre ELSE y el bloque en " + ubicacion(ultimoError)
        );
        RESULT = new NodoElse(b, tokelseleft, bright);
        :}
        | ELSE:tokelse bloque:b
        {: RESULT = new NodoElse(b, tokelseleft, bright); :}
        |
        {: RESULT = new NodoFinIf(); :}
        ;

sentencia_while ::=
```

```
	WHILE:tokw PARENTESIS_APERTURA:pa expresion:cond
PARENTESIS_CIERRE:pc bloque:b
	{: RESULT = new NodoWhile(cond, b, tokwleft, bright); :}
	| WHILE:tokw PARENTESIS_APERTURA:pa expresion:cond
PARENTESIS_CIERRE:pc error bloque:b
	{:
	erroresSintacticos.add(
	"Se ignoró contenido inválido entre WHILE y el bloque en " + ubicacion(ultimoError)
	);
	RESULT = new NodoWhile(cond, b, tokwleft, bright);
	:}
	;

sentencia_do_while ::=
	DO:tokdo bloque:b WHILE:tokw PARENTESIS_APERTURA:pa expresion:cond
PARENTESIS_CIERRE:pc
	{: RESULT = new NodoDoWhile(b, cond, tokdoleft, pcright); :}
	;

sentencia_for ::=
	sentencia_for_clasico:s
	{: RESULT = s; :}
	| sentencia_for_rango:s
	{: RESULT = s; :}
	;

sentencia_for_clasico ::=
	FOR:tokfor PARENTESIS_APERTURA:pa inicializacion_for:init
PUNTO_Y_COMA:sc1 condicion_for:cond PUNTO_Y_COMA:sc2 actualizacion_for:act
PARENTESIS_CIERRE:pc bloque:b
	{: RESULT = new NodoForClasico(init, cond, act, b, tokforleft, bright); :}
	| FOR:tokfor PARENTESIS_APERTURA:pa inicializacion_for:init
PUNTO_Y_COMA:sc1 condicion_for:cond PUNTO_Y_COMA:sc2 actualizacion_for:act
PARENTESIS_CIERRE:pc error bloque:b
	{:
	erroresSintacticos.add(
	"Se ignoró contenido inválido entre FOR y el bloque en " + ubicacion(ultimoError)
	);
	RESULT = new NodoForClasico(init, cond, act, b, tokforleft, bright);
	:}
	;

inicializacion_for ::=
	IDENTIFICADOR:id ASIGNACION:a expresion:e
	{: RESULT = new NodoAsignacion(id, e, idleft, eright); :}
	;

condicion_for ::=
```

```
	expresion:e
	{: RESULT = e; :}
	;

actualizacion_for ::=
	IDENTIFICADOR:id ASIGNACION:a expresion:e
	{: RESULT = new NodoAsignacion(id, e, idleft, eright); :}
	;

sentencia_for_rango ::=
	FOR:tokfor PARENTESIS_APERTURA:pa IDENTIFICADOR:id IN:tokin
expresion:inicio PUNTO:p1 PUNTO:p2 expresion:fin PARENTESIS_CIERRE:pc bloque:b
	{: RESULT = new NodoForRango(id, inicio, fin, b, tokforleft, bright); :}
	| FOR:tokfor PARENTESIS_APERTURA:pa IDENTIFICADOR:id IN:tokin
expresion:inicio PUNTO:p1 PUNTO:p2 expresion:fin PARENTESIS_CIERRE:pc error
bloque:b
	{:
	erroresSintacticos.add(
	"Se ignoró contenido inválido entre FOR rango y el bloque en " +
ubicacion(ultimoError)
	);
	RESULT = new NodoForRango(id, inicio, fin, b, tokforleft, bright);
	:}
	;
```

```
package com.mycompany.testlexer2;

import java_cup.runtime.*;
import java.util.ArrayList;
import java.util.List;

import com.mycompany.testlexer2.ast.*;

parser code {:
	private final List<String> erroresSintacticos = new ArrayList<>();
	public Symbol ultimoError = null;

	public List<String> getErroresSintacticos(){
	return erroresSintacticos;
	}

	private int linea(Symbol s) {
	return s == null ? -1 : s.left + 1;
```

```
}

private int columna(Symbol s) {
return s == null ? -1 : s.right + 1;
}

private String ubicacion(Symbol s) {
if (s == null) return "fin de entrada";
return "línea " + linea(s) + ", columna " + columna(s);
}

@Override
public void report_error(String message, Object info) {
if (info instanceof Symbol s) {
ultimoError = s;
erroresSintacticos.add(
        "Error sintáctico en " + ubicacion(s) + ": " + message
);
} else {
erroresSintacticos.add("Error sintáctico: " + message);
}
}

@Override
public void syntax_error(Symbol cur_token) {
ultimoError = cur_token;
if (cur_token != null) {
erroresSintacticos.add(
        "Token inesperado en " + ubicacion(cur_token)
);
} else {
erroresSintacticos.add("Error sintáctico: fin de entrada inesperado");
}
}

@Override
public void unrecovered_syntax_error(Symbol cur_token) throws Exception {
ultimoError = cur_token;
if (cur_token != null) {
erroresSintacticos.add(
        "No se pudo recuperar del error en " + ubicacion(cur_token)
);
} else {
erroresSintacticos.add(
        "No se pudo recuperar del error al final de la entrada"
);
}
```

```
	throw new Exception("Error sintáctico: " + (ultimoError.left + 1) + ":" +
(ultimoError.right + 1));
	}
:};

scan with {:
	return getScanner().next_token();
:};


terminal String ENTERO;

terminal String CADENA;

terminal String COLOR_HEXADECIMAL;
terminal String COLOR_RGB;
terminal String COLOR_HSL;

terminal RESTA, DIVISION;

terminal PARENTESIS_APERTURA, PARENTESIS_CIERRE;
terminal LLAVE_APERTURA, LLAVE_CIERRE;

terminal MAYOR_QUE, MENOR_QUE;

terminal STYLE, COLOR, BACKGROUND, FONT, FAMILY;
terminal TEXT, SIZE, SECTION, CONTENT;
terminal TABLE, LINE, ELEMENT;

terminal OPEN, DROP, SELECT, MULTIPLE;

terminal VERTICAL, HORIZONTAL;

terminal String COLOR_PREDEFINIDO;
terminal String FUENTE_PREDEFINIDA;
terminal String TIPO_BORDE;
terminal String BORDER;

terminal COMA, ASIGNACION;

terminal EMOJI_RISA, EMOJI_TRISTE, EMOJI_SERIO, EMOJI_CORAZON,
EMOJI_ESTRELLA, EMOJI_GATO;

non terminal NodoPrograma programa;

non terminal List<NodoElemento> lista_elementos;
non terminal NodoElemento elemento;
```

```
non terminal NodoEstilos estilo;
non terminal List<NodoEstilo> lista_estilos;
non terminal NodoEstilo estilo_item;
non terminal NodoExpresion valor_color;

non terminal NodoSeccion seccion;
non terminal NodoLiteral orientacion;
non terminal List<NodoAtributo> seccion_cuerpo;

non terminal List<NodoElemento> contenido;
non terminal List<NodoElemento> lista_contenido;
non terminal NodoElemento item_contenido;

non terminal NodoTabla tabla;

non terminal NodoFilasTabla contenido_tabla;
non terminal List<List<NodoElemento>> lista_lineas;
non terminal List<NodoElemento> linea;

non terminal List<NodoElemento> lista_elementos_tabla;
non terminal NodoElemento elemento_tabla;

non terminal NodoExpresion contenido_elemento;
non terminal NodoLiteral contenido_texto;
non terminal NodoExpresion contenido_unitario;

non terminal NodoLiteral emoji;

non terminal NodoTexto texto;

non terminal NodoPregunta pregunta;
non terminal NodoPreguntaAbierta pregunta_open;
non terminal NodoPreguntaDesplegable pregunta_drop;
non terminal NodoPreguntaSeleccionUnica pregunta_select;
non terminal NodoPreguntaSeleccionMultiple pregunta_multiple;

non terminal cierre_simple;

non terminal List<NodoExpresion> opciones;
non terminal List<NodoExpresion> lista_opciones;

non terminal List<Integer> respuestas;
non terminal List<Integer> lista_respuestas;


start with programa;

programa ::=
```

```
  lista_elementos:l
  {: RESULT = new NodoPrograma(l, true, 1, 1); :}
        ;

lista_elementos ::=
        lista_elementos:l elemento:e
        {: l.add(e); RESULT = l; :}
        | elemento:e
        {:
        List<NodoElemento> tmp = new ArrayList<>();
        tmp.add(e);
        RESULT = tmp;
        :}
        ;

elemento ::=
        seccion:s {: RESULT = s; :}
        | tabla:t {: RESULT = t; :}
        | pregunta:p {: RESULT = p; :}
        | texto:tx {: RESULT = tx; :}
        ;

estilo ::=
  MENOR_QUE STYLE MAYOR_QUE lista_estilos:l MENOR_QUE DIVISION STYLE
MAYOR_QUE
  {: RESULT = new NodoEstilos(l, 1, 1); :}
        ;

lista_estilos ::=
        lista_estilos:l estilo_item:it
        {: l.add(it); RESULT = l; :}
        | estilo_item:it
        {:
        List<NodoEstilo> tmp = new ArrayList<>();
        tmp.add(it);
        RESULT = tmp;
        :}
        ;

estilo_item ::=
        MENOR_QUE COLOR ASIGNACION valor_color:v DIVISION MAYOR_QUE
        {: RESULT = new NodoEstilo("color", v, 1, 1); :}
        | MENOR_QUE BACKGROUND COLOR ASIGNACION valor_color:v DIVISION
MAYOR_QUE
        {: RESULT = new NodoEstilo("background-color", v, 1, 1); :}
        | MENOR_QUE FONT FAMILY ASIGNACION FUENTE_PREDEFINIDA:f DIVISION
MAYOR_QUE
        {: RESULT = new NodoEstilo("font-family", new NodoLiteral("string", f, 1, 1), 1, 1); :}
```

```
        | MENOR_QUE TEXT SIZE ASIGNACION ENTERO:n DIVISION MAYOR_QUE
        {: RESULT = new NodoEstilo("text-size", new NodoLiteral("number", n, 1, 1), 1, 1); :}
        | MENOR_QUE BORDER ASIGNACION ENTERO:n COMA TIPO_BORDE:t COMA
COLOR ASIGNACION valor_color:v DIVISION MAYOR_QUE
        {: RESULT = new NodoBorde(
        "border",
        new NodoLiteral("number", n, 1, 1),
        new NodoLiteral("tipo_borde", (String)t, 1, 1),
        v,
        1,
        1); :}
        | MENOR_QUE BORDER ASIGNACION ENTERO:n COMA LINE:t COMA COLOR
ASIGNACION valor_color:v DIVISION MAYOR_QUE
        {: RESULT = new NodoBorde("border", new NodoLiteral("number", n, 1, 1), new
NodoLiteral("tipo_borde", (String)t, 1, 1), v, 1, 1); :}
        ;

valor_color ::=
        COLOR_HEXADECIMAL:s {: RESULT = new NodoLiteral("color_hex", s, 1, 1); :}
        | COLOR_RGB:s {: RESULT = new NodoLiteral("color_rgb", s.substring(3), 1, 1); :}
        | COLOR_HSL:s {: RESULT = new NodoLiteral("color_hsl", s, 1, 1); :}
        | COLOR_PREDEFINIDO:s {: RESULT = new NodoLiteral("color_name", s, 1, 1); :}
        ;

seccion ::=
        MENOR_QUE SECTION ASIGNACION ENTERO:w COMA ENTERO:h COMA
ENTERO:px COMA ENTERO:py COMA orientacion:o MAYOR_QUE
        seccion_cuerpo:attrs
        MENOR_QUE DIVISION SECTION MAYOR_QUE
  {:
        attrs.add(0, new NodoAtributo("orientation", o, 1, 1));
        attrs.add(0, new NodoAtributo("pointY", new NodoLiteral("number", py, 1, 1), 1, 1));
        attrs.add(0, new NodoAtributo("pointX", new NodoLiteral("number", px, 1, 1), 1, 1));
        attrs.add(0, new NodoAtributo("height", new NodoLiteral("number", h, 1, 1), 1, 1));
        attrs.add(0, new NodoAtributo("width", new NodoLiteral("number", w, 1, 1), 1, 1));

        RESULT = new NodoSeccion(attrs, 1, 1);
  :}
        ;

orientacion ::=
        VERTICAL {: RESULT = new NodoLiteral("orientacion", "vertical", 1, 1); :}
        | HORIZONTAL {: RESULT = new NodoLiteral("orientacion", "horizontal", 1, 1); :}
        ;

seccion_cuerpo ::=
  estilo:st contenido:els
  {:
```

```
        List<NodoAtributo> attrs = new ArrayList<>();
        attrs.add(new NodoAtributo("estilos", st, 1, 1));
        attrs.add(new NodoAtributo("contenido", new NodoListaElementos(els, 1, 1), 1, 1));
        RESULT = attrs;
  :}
        ;

contenido ::=
  MENOR_QUE CONTENT MAYOR_QUE lista_contenido:l MENOR_QUE DIVISION
CONTENT MAYOR_QUE
  {: RESULT = l; :}
        ;


lista_contenido ::=
        lista_contenido:l item_contenido:i
        {: l.add(i); RESULT = l; :}
        | item_contenido:i
        {:
        List<NodoElemento> tmp = new ArrayList<>();
        tmp.add(i);
        RESULT = tmp;
        :}
        ;

item_contenido ::=
        texto:t {: RESULT = t; :}
        | tabla:tb {: RESULT = tb; :}
        | pregunta:p {: RESULT = p; :}
        | seccion:s {: RESULT = s; :}
        ;

tabla ::=
        MENOR_QUE TABLE ASIGNACION ENTERO:w COMA ENTERO:h COMA
ENTERO:px COMA ENTERO:py MAYOR_QUE
  estilo:st
        contenido_tabla:filas
        MENOR_QUE DIVISION TABLE MAYOR_QUE
  {:
        List<NodoAtributo> attrs = new ArrayList<>();
        attrs.add(new NodoAtributo("estilos", st, 1, 1));
        attrs.add(new NodoAtributo("filas", filas, 1, 1));
        attrs.add(0, new NodoAtributo("pointY", new NodoLiteral("number", py, 1, 1), 1, 1));
        attrs.add(0, new NodoAtributo("pointX", new NodoLiteral("number", px, 1, 1), 1, 1));
        attrs.add(0, new NodoAtributo("height", new NodoLiteral("number", h, 1, 1), 1, 1));
        attrs.add(0, new NodoAtributo("width", new NodoLiteral("number", w, 1, 1), 1, 1));
        RESULT = new NodoTabla(attrs, 1, 1);
  :}
```

```
        ;

contenido_tabla ::=
        MENOR_QUE CONTENT MAYOR_QUE lista_lineas:ls MENOR_QUE DIVISION
CONTENT MAYOR_QUE
        {:
        RESULT = new NodoFilasTabla(ls, 1, 1);
        :}
        ;

lista_lineas ::=
        lista_lineas:l linea:ln
        {: l.add(ln); RESULT = l; :}
        | linea:ln
        {:
        List<List<NodoElemento>> tmp = new ArrayList<>();
        tmp.add(ln);
        RESULT = tmp;
        :}
        ;

linea ::=
  MENOR_QUE LINE MAYOR_QUE lista_elementos_tabla:els MENOR_QUE DIVISION
LINE MAYOR_QUE
  {: RESULT = els; :}
        ;

lista_elementos_tabla ::=
        lista_elementos_tabla:l elemento_tabla:e
        {: l.add(e); RESULT = l; :}
        | elemento_tabla:e
        {:
        List<NodoElemento> tmp = new ArrayList<>();
        tmp.add(e);
        RESULT = tmp;
        :}
        ;

elemento_tabla ::=
  MENOR_QUE ELEMENT MAYOR_QUE elemento:e MENOR_QUE DIVISION ELEMENT
MAYOR_QUE
  {: RESULT = e; :}
;


contenido_elemento ::=
        contenido_elemento:acc contenido_unitario:part
        {:
```

```
        List<NodoExpresion> partes = new ArrayList<>();
        if (acc instanceof NodoCadenaCompuesta cc) {
        partes.addAll(cc.getPartes());
        } else {
        partes.add(acc);
        }
        partes.add(part);
        RESULT = new NodoCadenaCompuesta(partes, 1, 1);
        :}
        | contenido_unitario:part
        {: RESULT = part; :}
        ;

contenido_unitario ::=
        contenido_texto:t {: RESULT = t; :}
        | emoji:e {: RESULT = e; :}
        ;


contenido_texto ::=
  CADENA:s {: RESULT = new NodoLiteral("string", s, 1, 1); :}
        ;


emoji ::=
        EMOJI_RISA {: RESULT = new NodoLiteral("emoji", "risa", 1, 1); :}
        | EMOJI_TRISTE {: RESULT = new NodoLiteral("emoji", "triste", 1, 1); :}
        | EMOJI_SERIO {: RESULT = new NodoLiteral("emoji", "serio", 1, 1); :}
        | EMOJI_CORAZON {: RESULT = new NodoLiteral("emoji", "corazon", 1, 1); :}
        | EMOJI_ESTRELLA {: RESULT = new NodoLiteral("emoji", "estrella", 1, 1); :}
        | EMOJI_GATO {: RESULT = new NodoLiteral("emoji", "gato", 1, 1); :}
        ;

texto ::=
        MENOR_QUE TEXT ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont cierre_simple
        {:
        List<NodoAtributo> attrs = new ArrayList<>();
        attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("contenido", cont, 1, 1));
        RESULT = new NodoTexto(attrs, 1, 1);
        :}
        | MENOR_QUE TEXT ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont MAYOR_QUE
        estilo:st
        MENOR_QUE DIVISION TEXT MAYOR_QUE
        {:
```

```
	List<NodoAtributo> attrs = new ArrayList<>();
	attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
	attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
	attrs.add(new NodoAtributo("contenido", cont, 1, 1));
	attrs.add(new NodoAtributo("estilos", st, 1, 1));
	RESULT = new NodoTexto(attrs, 1, 1);
	:}
	| MENOR_QUE TEXT ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont MAYOR_QUE
	MENOR_QUE DIVISION TEXT MAYOR_QUE
	{:
	List<NodoAtributo> attrs = new ArrayList<>();
	attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
	attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
	attrs.add(new NodoAtributo("contenido", cont, 1, 1));
	RESULT = new NodoTexto(attrs, 1, 1);
	:}
;

pregunta ::=
	pregunta_open:p {: RESULT = p; :}
	| pregunta_drop:p {: RESULT = p; :}
	| pregunta_select:p {: RESULT = p; :}
	| pregunta_multiple:p {: RESULT = p; :}
	;

pregunta_open ::=
	MENOR_QUE OPEN ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont cierre_simple
	{:
	List<NodoAtributo> attrs = new ArrayList<>();
	attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
	attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
	attrs.add(new NodoAtributo("contenido", cont, 1, 1));
	RESULT = new NodoPreguntaAbierta(attrs, 1, 1);
	:}
	| MENOR_QUE OPEN ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont MAYOR_QUE
	estilo:st
	MENOR_QUE DIVISION OPEN MAYOR_QUE
	{:
	List<NodoAtributo> attrs = new ArrayList<>();
	attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
	attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
	attrs.add(new NodoAtributo("contenido", cont, 1, 1));
	attrs.add(new NodoAtributo("estilos", st, 1, 1));
	RESULT = new NodoPreguntaAbierta(attrs, 1, 1);
	:}
```

```
        | MENOR_QUE OPEN ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont MAYOR_QUE
        MENOR_QUE DIVISION OPEN MAYOR_QUE
        {:
        List<NodoAtributo> attrs = new ArrayList<>();
        attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("contenido", cont, 1, 1));
        RESULT = new NodoPreguntaAbierta(attrs, 1, 1);
        :}
        ;

pregunta_drop ::=
        MENOR_QUE DROP ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont COMA opciones:ops COMA ENTERO:resp cierre_simple
        {:
        List<NodoAtributo> attrs = new ArrayList<>();
        attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("contenido", cont, 1, 1));
        attrs.add(new NodoAtributo("opciones", new NodoCadenaCompuesta(ops, 1, 1), 1,
1));
        attrs.add(new NodoAtributo("respuesta", new NodoLiteral("number", resp, 1, 1), 1,
1));
        RESULT = new NodoPreguntaDesplegable(attrs, 1, 1);
        :}
        | MENOR_QUE DROP ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont COMA opciones:ops COMA RESTA ENTERO:resp cierre_simple
        {:
        List<NodoAtributo> attrs = new ArrayList<>();
        attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("contenido", cont, 1, 1));
        attrs.add(new NodoAtributo("opciones", new NodoCadenaCompuesta(ops, 1, 1), 1,
1));
        attrs.add(new NodoAtributo("respuesta", new NodoLiteral("number", "-" + resp, 1, 1),
1, 1));
        RESULT = new NodoPreguntaDesplegable(attrs, 1, 1);
        :}
        | MENOR_QUE DROP ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont COMA opciones:ops COMA ENTERO:resp MAYOR_QUE
        estilo:st
        MENOR_QUE DIVISION DROP MAYOR_QUE
        {:
        List<NodoAtributo> attrs = new ArrayList<>();
        attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("contenido", cont, 1, 1));
```

```
        attrs.add(new NodoAtributo("opciones", new NodoCadenaCompuesta(ops, 1, 1), 1,
1));
        attrs.add(new NodoAtributo("respuesta", new NodoLiteral("number", resp, 1, 1), 1,
1));
        attrs.add(new NodoAtributo("estilos", st, 1, 1));
        RESULT = new NodoPreguntaDesplegable(attrs, 1, 1);
        :}
        | MENOR_QUE DROP ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont COMA opciones:ops COMA RESTA ENTERO:resp MAYOR_QUE
        estilo:st
        MENOR_QUE DIVISION DROP MAYOR_QUE
        {:
        List<NodoAtributo> attrs = new ArrayList<>();
        attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("contenido", cont, 1, 1));
        attrs.add(new NodoAtributo("opciones", new NodoCadenaCompuesta(ops, 1, 1), 1,
1));
        attrs.add(new NodoAtributo("respuesta", new NodoLiteral("number", "-" + resp, 1, 1),
1, 1));
        attrs.add(new NodoAtributo("estilos", st, 1, 1));
        RESULT = new NodoPreguntaDesplegable(attrs, 1, 1);
        :}
        | MENOR_QUE DROP ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont COMA opciones:ops COMA RESTA ENTERO:resp MAYOR_QUE
        MENOR_QUE DIVISION DROP MAYOR_QUE
        {:
        List<NodoAtributo> attrs = new ArrayList<>();
        attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("contenido", cont, 1, 1));
        attrs.add(new NodoAtributo("opciones", new NodoCadenaCompuesta(ops, 1, 1), 1,
1));
        attrs.add(new NodoAtributo("respuesta", new NodoLiteral("number", "-" + resp, 1, 1),
1, 1));
        RESULT = new NodoPreguntaDesplegable(attrs, 1, 1);
        :}
        | MENOR_QUE DROP ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont COMA opciones:ops COMA ENTERO:resp MAYOR_QUE
        MENOR_QUE DIVISION DROP MAYOR_QUE
        {:
        List<NodoAtributo> attrs = new ArrayList<>();
        attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("contenido", cont, 1, 1));
        attrs.add(new NodoAtributo("opciones", new NodoCadenaCompuesta(ops, 1, 1), 1,
1));
```

```
        attrs.add(new NodoAtributo("respuesta", new NodoLiteral("number", resp, 1, 1), 1,
1));
        RESULT = new NodoPreguntaDesplegable(attrs, 1, 1);
        :}
        ;

pregunta_select ::=
        MENOR_QUE SELECT ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont COMA opciones:ops COMA ENTERO:resp cierre_simple
        {:
        List<NodoAtributo> attrs = new ArrayList<>();
        attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("contenido", cont, 1, 1));
        attrs.add(new NodoAtributo("opciones", new NodoCadenaCompuesta(ops, 1, 1), 1,
1));
        attrs.add(new NodoAtributo("respuesta", new NodoLiteral("number", resp, 1, 1), 1,
1));
        RESULT = new NodoPreguntaSeleccionUnica(attrs, 1, 1);
        :}
        | MENOR_QUE SELECT ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont COMA opciones:ops COMA ENTERO:resp MAYOR_QUE
        estilo:st
        MENOR_QUE DIVISION SELECT MAYOR_QUE
        {:
        List<NodoAtributo> attrs = new ArrayList<>();
        attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("contenido", cont, 1, 1));
        attrs.add(new NodoAtributo("opciones", new NodoCadenaCompuesta(ops, 1, 1), 1,
1));
        attrs.add(new NodoAtributo("respuesta", new NodoLiteral("number", resp, 1, 1), 1,
1));
        attrs.add(new NodoAtributo("estilos", st, 1, 1));
        RESULT = new NodoPreguntaSeleccionUnica(attrs, 1, 1);
        :}
        | MENOR_QUE SELECT ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont COMA opciones:ops COMA ENTERO:resp MAYOR_QUE
        MENOR_QUE DIVISION SELECT MAYOR_QUE
        {:
        List<NodoAtributo> attrs = new ArrayList<>();
        attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("contenido", cont, 1, 1));
        attrs.add(new NodoAtributo("opciones", new NodoCadenaCompuesta(ops, 1, 1), 1,
1));
        attrs.add(new NodoAtributo("respuesta", new NodoLiteral("number", resp, 1, 1), 1,
1));
```

```
        RESULT = new NodoPreguntaSeleccionUnica(attrs, 1, 1);
        :}
        ;

pregunta_multiple ::=
        MENOR_QUE MULTIPLE ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont COMA opciones:ops COMA respuestas:res cierre_simple
        {:
        List<NodoAtributo> attrs = new ArrayList<>();
        attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("contenido", cont, 1, 1));
        attrs.add(new NodoAtributo("opciones", new NodoCadenaCompuesta(ops, 1, 1), 1,
1));
        attrs.add(new NodoAtributo("respuestas", new NodoLiteral("lista_int", res.toString(),
1, 1), 1, 1));
        RESULT = new NodoPreguntaSeleccionMultiple(attrs, 1, 1);
        :}
        | MENOR_QUE MULTIPLE ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont COMA opciones:ops COMA respuestas:res MAYOR_QUE
        estilo:st
        MENOR_QUE DIVISION MULTIPLE MAYOR_QUE
        {:
        List<NodoAtributo> attrs = new ArrayList<>();
        attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("contenido", cont, 1, 1));
        attrs.add(new NodoAtributo("opciones", new NodoCadenaCompuesta(ops, 1, 1), 1,
1));
        attrs.add(new NodoAtributo("respuestas", new NodoLiteral("lista_int", res.toString(),
1, 1), 1, 1));
        attrs.add(new NodoAtributo("estilos", st, 1, 1));
        RESULT = new NodoPreguntaSeleccionMultiple(attrs, 1, 1);
        :}
        | MENOR_QUE MULTIPLE ASIGNACION ENTERO:x COMA ENTERO:y COMA
contenido_elemento:cont COMA opciones:ops COMA respuestas:res MAYOR_QUE
        MENOR_QUE DIVISION MULTIPLE MAYOR_QUE
        {:
        List<NodoAtributo> attrs = new ArrayList<>();
        attrs.add(new NodoAtributo("x", new NodoLiteral("number", x, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("y", new NodoLiteral("number", y, 1, 1), 1, 1));
        attrs.add(new NodoAtributo("contenido", cont, 1, 1));
        attrs.add(new NodoAtributo("opciones", new NodoCadenaCompuesta(ops, 1, 1), 1,
1));
        attrs.add(new NodoAtributo("respuestas", new NodoLiteral("lista_int", res.toString(),
1, 1), 1, 1));
        RESULT = new NodoPreguntaSeleccionMultiple(attrs, 1, 1);
        :}
```

```
        ;

cierre_simple ::=
        DIVISION MAYOR_QUE
        ;

opciones ::=
  LLAVE_APERTURA lista_opciones:l LLAVE_CIERRE
  {: RESULT = l; :}
        ;

lista_opciones ::=
        lista_opciones:l COMA contenido_elemento:e
        {:
        l.add(e);
        RESULT = l;
        :}
        | contenido_elemento:e
        {:
        List<NodoExpresion> tmp = new ArrayList<>();
        tmp.add(e);
        RESULT = tmp;
        :}
        ;

respuestas ::=
        LLAVE_APERTURA lista_respuestas:l LLAVE_CIERRE
        {: RESULT = l; :}
        | LLAVE_APERTURA LLAVE_CIERRE
        {:
        RESULT = new ArrayList<>();
        :}
        ;

lista_respuestas ::=
        lista_respuestas:l COMA ENTERO:n
        {:
        l.add(Integer.parseInt(n));
        RESULT = l;
        :}
        | ENTERO:n
        {:
        List<Integer> tmp = new ArrayList<>();
        tmp.add(Integer.parseInt(n));
        RESULT = tmp;
        :}
        ;
```